

```
}  
}
```

Receive a notification when all frames have been captured

The HDR photo capture has two steps. First, multiple frames are captured, and then the frames are processed into the final HDR image. You can't initiate another capture while the source HDR frames are still being captured, but you can initiate a capture after all of the frames have been captured but before the HDR post-processing is complete. The [AllPhotosCaptured](#) event is raised when the HDR captures are complete, letting you know that you can initiate another capture. A typical scenario is to disable your UI's capture button when HDR capture begins and then reenable it when [AllPhotosCaptured](#) is raised.

C#

```
private void AdvancedCapture_AllPhotosCaptured(AdvancedPhotoCapture sender,  
object args)  
{  
    // Update UI to enable capture button  
}
```

Clean up the AdvancedPhotoCapture object

When your app is done capturing, before disposing of the **MediaCapture** object, you should shut down the [AdvancedPhotoCapture](#) object by calling [FinishAsync](#) and setting your member variable to null.

C#

```
await _advancedCapture.FinishAsync();  
_advancedCapture = null;
```

Low-light photo capture

Starting with Windows 10, version 1607, **AdvancedPhotoCapture** can be used to capture photos using a built-in algorithm that enhances the quality of photos captured in low-light settings. When you use the low-light feature of the [AdvancedPhotoCapture](#) class, the system will evaluate the current scene and, if needed, apply an algorithm to

compensate for low-light conditions. If the system determines that the algorithm is not needed, a regular capture is performed instead.

Before using low-light photo capture, determine if the device on which your app is currently running supports the technique by getting the **MediaCapture** object's **VideoDeviceController** and then getting the **AdvancedPhotoControl** property. Check the video device controller's **SupportedModes** collection to see if it includes **AdvancedPhotoMode.LowLight**. If it does, low-light capture using **AdvancedPhotoCapture** is supported.

C#

```
bool _lowLightSupported;
```

C#

```
_lowLightSupported =  
_mediaCapture.VideoDeviceController.AdvancedPhotoControl.SupportedModes.Contains(Windows.Media.Devices.AdvancedPhotoMode.LowLight);
```

Next, declare a member variable to store the **AdvancedPhotoCapture** object.

C#

```
private AdvancedPhotoCapture _advancedCapture;
```

In your app, after you have initialized the **MediaCapture** object, create an **AdvancedPhotoCaptureSettings** object and set the mode to **AdvancedPhotoMode.LowLight**. Call the **AdvancedPhotoControl** object's **Configure** method, passing in the **AdvancedPhotoCaptureSettings** object you created.

Call the **MediaCapture** object's **PrepareAdvancedPhotoCaptureAsync**, passing in an **ImageEncodingProperties** object specifying the type of encoding the capture should use.

C#

```
if (_lowLightSupported == false) return;  
  
// Choose LowLight mode  
var settings = new AdvancedPhotoCaptureSettings { Mode =  
AdvancedPhotoMode.LowLight };  
_mediaCapture.VideoDeviceController.AdvancedPhotoControl.Configure(settings)  
;
```

```
// Prepare for an advanced capture
_advancedCapture =
    await
_mediaCapture.PrepareAdvancedPhotoCaptureAsync(ImageEncodingProperties.CreateUncompressed(MediaPixelFormat.Nv12));
```

To capture a photo, call [CaptureAsync](#).

C#

```
AdvancedCapturedPhoto advancedCapturedPhoto = await
_advancedCapture.CaptureAsync();
var photoOrientation =
ConvertOrientationToPhotoOrientation(GetCameraOrientation());
var fileName = String.Format("SimplePhoto_{0}_LowLight.jpg",
DateTime.Now.ToString("HHmmss"));
await SaveCapturedFrameAsync(advancedCapturedPhoto.Frame, fileName,
photoOrientation);
```

Like the HDR example above, this example uses a helper class called **CameraRotationHelper** to determine the rotation value that should be encoded into the image so that it can be displayed properly by other apps and devices. This class is described and listed in full in the article [Handle device orientation with MediaCapture](#).

The **SaveCapturedFrameAsync** helper method, which saves the image to disk, is discussed later in this article.

You can capture multiple low-light photos without reconfiguring the **AdvancedPhotoCapture** object, but when you are done capturing, you should call [FinishAsync](#) to clean up the object and associated resources.

C#

```
await _advancedCapture.FinishAsync();
_advancedCapture = null;
```

Working with AdvancedCapturedPhoto objects

[AdvancedPhotoCapture.CaptureAsync](#) returns an **AdvancedCapturedPhoto** object representing the captured photo. This object exposes the **Frame** property which returns a **CapturedFrame** object representing the image. The [OptionalReferencePhotoCaptured](#) event also provides a **CapturedFrame** object in its event args. After you get an object of this type, there are a number of things you can do with it, including creating a [SoftwareBitmap](#) or saving the image to a file.

Get a SoftwareBitmap from a CapturedFrame

It's trivial to get a **SoftwareBitmap** from a **CapturedFrame** object by simply accessing the **SoftwareBitmap** property of the object. However, most encoding formats do not support **SoftwareBitmap** with **AdvancedPhotoCapture**, so you should check and make sure the property is not null before using it.

C#

```
SoftwareBitmap bitmap;  
if (advancedCapturedPhoto.Frame.SoftwareBitmap != null)  
{  
    bitmap = advancedCapturedPhoto.Frame.SoftwareBitmap;  
}
```

In the current release, the only encoding format that supports **SoftwareBitmap** for **AdvancedPhotoCapture** is uncompressed NV12. So, if you want to use this feature, you must specify that encoding when you call **PrepareAdvancedPhotoCaptureAsync**.

C#

```
_advancedCapture =  
    await  
_mediaCapture.PrepareAdvancedPhotoCaptureAsync(ImageEncodingProperties.CreateUncompressed(MediaPixelFormat.Nv12));
```

Of course, you can always save the image to a file and then load the file into a **SoftwareBitmap** in a separate step. For more information about working with **SoftwareBitmap**, see [Create, edit, and save bitmap images](#).

Save a CapturedFrame to a file

The **CapturedFrame** class implements the **IInputStream** interface, so it can be used as the input to a **BitmapDecoder**, and then a **BitmapEncoder** can be used to write the image data to disk.

In the following example, a new folder in the user's pictures library is created and a file is created within this folder. Note that your app will need to include the **Pictures Library** capability in your app manifest file in order to access this directory. A file stream is then opened to the specified file. Next, the **BitmapDecoder.CreateAsync** is called to create the decoder from the **CapturedFrame**. Then **CreateForTranscodingAsync** creates an encoder from the file stream and the decoder.

The next steps encode the orientation of the photo into the image file by using the [BitmapProperties](#) of the encoder. For more information about handling orientation when capturing images, see [Handle device orientation with MediaCapture](#).

Finally, the image is written to the file with a call to [FlushAsync](#).

C#

```
private static async Task<StorageFile> SaveCapturedFrameAsync(CapturedFrame
frame, string fileName, PhotoOrientation photoOrientation)
{
    var folder = await
KnownFolders.PicturesLibrary.CreateFolderAsync("MyApp",
CreationCollisionOption.OpenIfExists);
    var file = await folder.CreateFileAsync(fileName,
CreationCollisionOption.GenerateUniqueName);

    using (var inputStream = frame)
    {
        using (var fileStream = await
file.OpenAsync(FileAccessMode.ReadWrite))
        {
            var decoder = await BitmapDecoder.CreateAsync(inputStream);
            var encoder = await
BitmapEncoder.CreateForTranscodingAsync(fileStream, decoder);
            var properties = new BitmapPropertySet {
                { "System.Photo.Orientation", new
BitmapTypedValue(photoOrientation, PropertyType.UInt16) } };
            await encoder.BitmapProperties.SetPropertiesAsync(properties);
            await encoder.FlushAsync();
        }
    }
    return file;
}
```

Related topics

- [Camera](#)
- [Basic photo, video, and audio capture with MediaCapture](#)

Manual camera controls for photo and video capture

Article • 10/20/2022

This article shows you how to use manual device controls to enable enhanced photo and video capture scenarios including optical image stabilization and smooth zoom.

The controls discussed in this article are all added to your app using the same pattern. First, check to see if the control is supported on the current device on which your app is running. If the control is supported, set the desired mode for the control. Typically, if a particular control is unsupported on the current device, you should disable or hide the UI element that allows the user to enable the feature.

The code in this article was adapted from the [Camera Manual Controls SDK sample](#). You can download the sample to see the code used in context or to use the sample as a starting point for your own app.

ⓘ Note

This article builds on concepts and code discussed in [Basic photo, video, and audio capture with MediaCapture](#), which describes the steps for implementing basic photo and video capture. We recommend that you familiarize yourself with the basic media capture pattern in that article before moving on to more advanced capture scenarios. The code in this article assumes that your app already has an instance of MediaCapture that has been properly initialized.

All of the device control APIs discussed in this article are members of the [Windows.Media.Devices](#) namespace.

C#

```
using Windows.Media.Devices;
```

Exposure

The [ExposureControl](#) allows you to set the shutter speed used during photo or video capture.

This example uses a [Slider](#) control to adjust the current exposure value and a checkbox to toggle automatic exposure adjustment.

XML

```
<Slider Name="ExposureSlider" ValueChanged="ExposureSlider_ValueChanged"/>
<TextBlock Name="ExposureTextBlock" Text="{Binding
ElementName=ExposureSlider,Path=Value}"/>
<CheckBox Name="ExposureAutoCheckBox" Content="Auto"
Checked="ExposureCheckBox_CheckedChanged"
Unchecked="ExposureCheckBox_CheckedChanged"/>
```

Check to see if the current capture device supports the **ExposureControl** by checking the [Supported](#) property. If the control is supported, you can show and enable the UI for this feature. Set the checked state of the checkbox to indicate if automatic exposure adjustment is currently active to the value of the [Auto](#) property.

The exposure value must be within the range supported by the device and must be an increment of the supported step size. Get the supported values for the current device by checking the [Min](#), [Max](#), and [Step](#) properties, which are used to set the corresponding properties of the slider control.

Set the slider control's value to the current value of the **ExposureControl** after unregistering the [ValueChanged](#) event handler so that the event is not triggered when the value is set.

C#

```
var exposureControl = _mediaCapture.VideoDeviceController.ExposureControl;

if (exposureControl.Supported)
{
    ExposureAutoCheckBox.Visibility = Visibility.Visible;
    ExposureSlider.Visibility = Visibility.Visible;

    ExposureAutoCheckBox.IsChecked = exposureControl.Auto;

    ExposureSlider.Minimum = exposureControl.Min.Ticks;
    ExposureSlider.Maximum = exposureControl.Max.Ticks;
    ExposureSlider.StepFrequency = exposureControl.Step.Ticks;

    ExposureSlider.ValueChanged -= ExposureSlider_ValueChanged;
    var value = exposureControl.Value;
    ExposureSlider.Value = value.Ticks;
    ExposureSlider.ValueChanged += ExposureSlider_ValueChanged;
}
else
{
    ExposureAutoCheckBox.Visibility = Visibility.Collapsed;
```

```
ExposureSlider.Visibility = Visibility.Collapsed;
}
```

In the **ValueChanged** event handler, get the current value of the control and then set the exposure value by calling **SetValueAsync**.

C#

```
private async void ExposureSlider_ValueChanged(object sender,
Windows.UI.Xaml.Controls.Primitives.RangeBaseValueChangedEventArgs e)
{
    var value = TimeSpan.FromTicks((long)(sender as Slider).Value);
    await
_mediaCapture.VideoDeviceController.ExposureControl.SetValueAsync(value);
}
```

In the **CheckedChanged** event handler of the auto exposure checkbox, turn automatic exposure adjustment on or off by calling **SetAutoAsync** and passing in a boolean value.

C#

```
private async void ExposureCheckBox_CheckedChanged(object sender,
RoutedEventArgs e)
{
    if(! _isPreviewing)
    {
        // Auto exposure only supported while preview stream is running.
        return;
    }

    var autoExposure = ((sender as CheckBox).IsChecked == true);
    await
_mediaCapture.VideoDeviceController.ExposureControl.SetAutoAsync(autoExposure);
}
```

Important

Automatic exposure mode is only supported while the preview stream is running. Check to make sure that the preview stream is running before turning on automatic exposure.

Exposure compensation

The **ExposureCompensationControl** allows you to set the exposure compensation used during photo or video capture.

This example uses a **Slider** control to adjust the current exposure compensation value.

XML

```
<Slider Name="EvSlider" ValueChanged="EvSlider_ValueChanged"/>
<TextBlock Text="{Binding ElementName=EvSlider,Path=Value}"
Name="EvTextBlock"/>
```

Check to see if the current capture device supports the **ExposureCompensationControl** by checking the **Supported** property. If the control is supported, you can show and enable the UI for this feature.

The exposure compensation value must be within the range supported by the device and must be an increment of the supported step size. Get the supported values for the current device by checking the **Min**, **Max**, and **Step** properties, which are used to set the corresponding properties of the slider control.

Set slider control's value to the current value of the **ExposureCompensationControl** after unregistering the **ValueChanged** event handler so that the event is not triggered when the value is set.

C#

```
var exposureCompensationControl =
_mediaCapture.VideoDeviceController.ExposureCompensationControl;

if (exposureCompensationControl.Supported)
{
    EvSlider.Visibility = Visibility.Visible;
    EvSlider.Minimum = exposureCompensationControl.Min;
    EvSlider.Maximum = exposureCompensationControl.Max;
    EvSlider.StepFrequency = exposureCompensationControl.Step;

    EvSlider.ValueChanged -= EvSlider_ValueChanged;
    EvSlider.Value = exposureCompensationControl.Value;
    EvSlider.ValueChanged += EvSlider_ValueChanged;
}
else
{
    EvSlider.Visibility = Visibility.Collapsed;
}
```

In the **ValueChanged** event handler, get the current value of the control and the set the exposure value by calling **SetValueAsync**.

C#

```
private async void EvSlider_ValueChanged(object sender,
Windows.UI.Xaml.Controls.Primitives.RangeBaseValueChangedEventArgs e)
{
    var value = (sender as Slider).Value;
    await
_mediaCapture.VideoDeviceController.ExposureCompensationControl.SetValueAsyn
c((float)value);
}
```

Flash

The **FlashControl** allows you to enable or disable the flash or to enable automatic flash, where the system dynamically determines whether to use the flash. This control also allows you to enable automatic red eye reduction on devices that support it. These settings all apply to capturing photos. The **TorchControl** is a separate control for turning the torch on or off for video capture.

This example uses a set of radio buttons to allow the user to switch between on, off, and auto flash settings. A checkbox is also provided to allow toggling of red eye reduction and the video torch.

XML

```
<RadioButton Name="FlashOnRadioButton" Content="On"
Checked="FlashOnRadioButton_Checked"/>
<RadioButton Name="FlashAutoRadioButton" Content="Auto"
Checked="FlashAutoRadioButton_Checked"/>
<RadioButton Name="FlashOffRadioButton" Content="Off"
Checked="FlashOffRadioButton_Checked"/>
<CheckBox Name="RedEyeFlashCheckBox" Content="Red Eye"
Visibility="Collapsed" Checked="RedEyeFlashCheckBox_CheckedChanged"
Unchecked="RedEyeFlashCheckBox_CheckedChanged"/>
<CheckBox Name="TorchCheckBox" Content="Video Light" Visibility="Collapsed"
Checked="TorchCheckBox_CheckedChanged"
Unchecked="TorchCheckBox_CheckedChanged"/>
```

Check to see if the current capture device supports the **FlashControl** by checking the **Supported** property. If the control is supported, you can show and enable the UI for this feature. If the **FlashControl** is supported, automatic red eye reduction may or may not be supported, so check the **RedEyeReductionSupported** property before enabling the UI. Because the **TorchControl** is separate from the flash control, you must also check its **Supported** property before using it.

In the **Checked** event handler for each of the flash radio buttons, enable or disable the appropriate corresponding flash setting. Note that to set the flash to always be used, you must set the **Enabled** property to true and the **Auto** property to false.

C#

```
var flashControl = _mediaCapture.VideoDeviceController.FlashControl;

if (flashControl.Supported)
{
    FlashAutoRadioButton.Visibility = Visibility.Visible;
    FlashOnRadioButton.Visibility = Visibility.Visible;
    FlashOffRadioButton.Visibility = Visibility.Visible;

    FlashAutoRadioButton.IsChecked = true;

    if (flashControl.RedEyeReductionSupported)
    {
        RedEyeFlashCheckBox.Visibility = Visibility.Visible;
    }

    // Video light is not strictly part of flash, but users might expect to find it there
    if (_mediaCapture.VideoDeviceController.TorchControl.Supported)
    {
        TorchCheckBox.Visibility = Visibility.Visible;
    }
}
else
{
    FlashAutoRadioButton.Visibility = Visibility.Collapsed;
    FlashOnRadioButton.Visibility = Visibility.Collapsed;
    FlashOffRadioButton.Visibility = Visibility.Collapsed;
}
```

C#

```
private void FlashOnRadioButton_Checked(object sender, RoutedEventArgs e)
{
    _mediaCapture.VideoDeviceController.FlashControl.Enabled = true;
    _mediaCapture.VideoDeviceController.FlashControl.Auto = false;
}

private void FlashAutoRadioButton_Checked(object sender, RoutedEventArgs e)
{
    _mediaCapture.VideoDeviceController.FlashControl.Enabled = true;
    _mediaCapture.VideoDeviceController.FlashControl.Auto = true;
}

private void FlashOffRadioButton_Checked(object sender, RoutedEventArgs e)
{
}
```

```
        _mediaCapture.VideoDeviceController.FlashControl.Enabled = false;
    }
```

In the handler for the red eye reduction checkbox, set the **RedEyeReduction** property to the appropriate value.

```
C#

private void RedEyeFlashCheckBox_CheckedChanged(object sender,
RoutedEventArgs e)
{
    _mediaCapture.VideoDeviceController.FlashControl.RedEyeReduction =
    (RedEyeFlashCheckBox.IsChecked == true);
}
```

Finally, in the handler for the video torch checkbox, set the **Enabled** property to the appropriate value.

```
C#

private void TorchCheckBox_CheckedChanged(object sender, RoutedEventArgs e)
{
    _mediaCapture.VideoDeviceController.TorchControl.Enabled =
    (TorchCheckBox.IsChecked == true);

    if(! (_isPreviewing && _isRecording))
    {
        System.Diagnostics.Debug.WriteLine("Torch may not emit light if
        preview and video capture are not running.");
    }
}
```

ⓘ Note

On some devices the torch will not emit light, even if **TorchControl.Enabled** is set to true, unless the device has a preview stream running and is actively capturing video. The recommended order of operations is to turn on the video preview, turn on the torch by setting **Enabled** to true, and then initiate video capture. On some devices the torch will light up after the preview is started. On other devices, the torch may not light up until video capture is started.

Focus

Three different commonly used methods for adjusting the focus of the camera are supported by the [FocusControl](#) object, continuous autofocus, tap to focus, and manual focus. A camera app may support all three of these methods, but for readability, this article discusses each technique separately. This section also discusses how to enable the focus assist light.

Continuous autofocus

Enabling continuous autofocus instructs the camera to adjust the focus dynamically to try to keep the subject of the photo or video in focus. This example uses a radio button to toggle continuous autofocus on and off.

XML

```
<RadioButton Content="CAF" Name="CafFocusRadioButton"
Checked="CafFocusRadioButton_Checked"/>
```

Check to see if the current capture device supports the **FocusControl** by checking the [Supported](#) property. Next, determine if continuous autofocus is supported by checking the [SupportedFocusModes](#) list to see if it contains the value [FocusMode.Continuous](#), and if so, show the continuous autofocus radio button.

C#

```
var focusControl = _mediaCapture.VideoDeviceController.FocusControl;

if (focusControl.Supported)
{
    CafFocusRadioButton.Visibility =
        focusControl.SupportedFocusModes.Contains(FocusMode.Continuous)
            ? Visibility.Visible : Visibility.Collapsed;
}
else
{
    CafFocusRadioButton.Visibility = Visibility.Collapsed;
}
```

In the [Checked](#) event handler for the continuous autofocus radio button, use the [VideoDeviceController.FocusControl](#) property to get an instance of the control. Call [UnlockAsync](#) to unlock the control in case your app has previously called [LockAsync](#) to enable one of the other focus modes.

Create a new [FocusSettings](#) object and set the [Mode](#) property to **Continuous**. Set the [AutoFocusRange](#) property to a value appropriate for your app scenario or selected by

the user from your UI. Pass your **FocusSettings** object into the **Configure** method, and then call **FocusAsync** to initiate continuous autofocus.

C#

```
private async void CafFocusRadioButton_Checked(object sender,
RoutedEventArgs e)
{
    if(! _isPreviewing)
    {
        // Autofocus only supported while preview stream is running.
        return;
    }

    var focusControl = _mediaCapture.VideoDeviceController.FocusControl;
    await focusControl.UnlockAsync();
    var settings = new FocusSettings { Mode = FocusMode.Continuous,
AutoFocusRange = AutoFocusRange.FullRange };
    focusControl.Configure(settings);
    await focusControl.FocusAsync();
}
```

Important

Autofocus mode is only supported while the preview stream is running. Check to make sure that the preview stream is running before turning on continuous autofocus.

Tap to focus

The tap-to-focus technique uses the **FocusControl** and the **RegionsOfInterestControl** to specify a subregion of the capture frame where the capture device should focus. The region of focus is determined by the user tapping on the screen displaying the preview stream.

This example uses a radio button to enable and disable tap-to-focus mode.

XML

```
<RadioButton Content="Tap" Name="TapFocusRadioButton"
Checked="TapFocusRadioButton_Checked"/>
```

Check to see if the current capture device supports the **FocusControl** by checking the **Supported** property. The **RegionsOfInterestControl** must be supported, and must support at least one region, in order to use this technique. Check the

[AutoFocusSupported](#) and [MaxRegions](#) properties to determine whether to show or hide the radio button for tap-to-focus.

C#

```
var focusControl = _mediaCapture.VideoDeviceController.FocusControl;

if (focusControl.Supported)
{
    TapFocusRadioButton.Visibility =
        (_mediaCapture.VideoDeviceController.RegionsOfInterestControl.AutoFocusSupported &&
         _mediaCapture.VideoDeviceController.RegionsOfInterestControl.MaxRegions > 0)
        ? Visibility.Visible :
        Visibility.Collapsed;
}
else
{
    TapFocusRadioButton.Visibility = Visibility.Collapsed;
}
```

In the [Checked](#) event handler for the tap-to-focus radio button, use the [VideoDeviceController.FocusControl](#) property to get an instance of the control. Call [LockAsync](#) to lock the control in case your app has previously called [UnlockAsync](#) to enable continuous autofocus, and then wait for the user to tap the screen to change the focus.

C#

```
private async void TapFocusRadioButton_Checked(object sender,
RoutedEventArgs e)
{
    // Lock focus in case Continuous Autofocus was active when switching to
    Tap-to-focus
    var focusControl = _mediaCapture.VideoDeviceController.FocusControl;
    await focusControl.LockAsync();
    // Wait for user tap
}
```

This example focuses on a region when the user taps the screen, and then removes the focus from that region when the user taps again, like a toggle. Use a boolean variable to track the current toggled state.

C#

```
bool _isFocused = false;
```

The next step is to listen for the event when the user taps the screen by handling the **Tapped** event of the **CaptureElement** that is currently displaying the capture preview stream. If the camera isn't currently previewing, or if tap-to-focus mode is disabled, return from the handler without doing anything.

If the tracking variable *_isFocused* is toggled to false, and if the camera isn't currently in the process of focus (determined by the **FocusState** property of the **FocusControl**), begin the tap-to-focus process. Get the position of the user's tap from the event args passed into the handler. This example also uses this opportunity to pick the size of the region that will be focused upon. In this case, the size is 1/4 of the smallest dimension of the capture element. Pass the tap position and the region size into the **TapToFocus** helper method that is defined in the next section.

If the *_isFocused* toggle is set to true, the user tap should clear the focus from the previous region. This is done in the **TapUnfocus** helper method shown below.

C#

```
private async void PreviewControl_Tapped(object sender,
TappedRoutedEventArgs e)
{
    if (!_isPreviewing || (TapFocusRadioButton.IsChecked != true)) return;

    if (!_isFocused &&
        _mediaCapture.VideoDeviceController.FocusControl.FocusState !=
        MediaCaptureFocusState.Searching)
    {
        var smallEdge = Math.Min(Window.Current.Bounds.Width,
            Window.Current.Bounds.Height);

        // Choose to make the focus rectangle 1/4th the length of the
        // shortest edge of the window
        var size = new Size(smallEdge / 4, smallEdge / 4);
        var position = e.GetPosition(sender as UIElement);

        // Note that at this point, a rect at "position" with size "size"
        // could extend beyond the preview area. The following method will reposition
        // the rect if that is the case
        await TapToFocus(position, size);
    }
    else
    {
        await TapUnfocus();
    }
}
```

In the **TapToFocus** helper method, first set the *_isFocused* toggle to true so that the next screen tap will release the focus from the tapped region.

The next task in this helper method is to determine the rectangle within the preview stream that will be assigned to the focus control. This requires two steps. The first step is to determine the rectangle that the preview stream takes up within the [CaptureElement](#) control. This depends on the dimensions of the preview stream and the orientation of the device. The helper method **GetPreviewStreamRectInControl**, shown at the end of this section, performs this task and returns the rectangle containing the preview stream.

The next task in **TapToFocus** is to convert the tap location and desired focus rectangle size, which were determined within the **CaptureElement.Tapped** event handler, into coordinates within capture stream. The **ConvertUiTapToPreviewRect** helper method, shown later in this section, performs this conversion and returns the rectangle, in capture stream coordinates, where the focus will be requested.

Now that the target rectangle has been obtained, create a new [RegionOfInterest](#) object, setting the [Bounds](#) property to the target rectangle obtained in the previous steps.

Get the capture device's [FocusControl](#). Create a new [FocusSettings](#) object and set the [Mode](#) and [AutoFocusRange](#) to your desired values, after checking to make sure that they are supported by the [FocusControl](#). Call [Configure](#) on the [FocusControl](#) to make your settings active and signal the device to begin focusing on the specified region.

Next, get the capture device's [RegionsOfInterestControl](#) and call [SetRegionsAsync](#) to set the active region. Multiple regions of interest can be set on devices that support it, but this example only sets a single region.

Finally, call [FocusAsync](#) on the [FocusControl](#) to initiate focusing.

Important

When implementing tap to focus, the order of operations is important. You should call these APIs in the following order:

1. [FocusControl.Configure](#)
2. [RegionsOfInterestControl.SetRegionsAsync](#)
3. [FocusControl.FocusAsync](#)

C#

```
public async Task TapToFocus(Point position, Size size)
{
    _isFocused = true;

    var previewRect = GetPreviewStreamRectInControl();
    var focusPreview = ConvertUiTapToPreviewRect(position, size,
```

```

previewRect);

    // Note that this Region Of Interest could be configured to also
    calculate exposure
    // and white balance within the region
    var regionOfInterest = new RegionOfInterest
    {
        AutoFocusEnabled = true,
        BoundsNormalized = true,
        Bounds = focusPreview,
        Type = RegionOfInterestType.Unknown,
        Weight = 100,
    };

    var focusControl = _mediaCapture.VideoDeviceController.FocusControl;
    var focusRange =
    focusControl.SupportedFocusRanges.Contains(AutoFocusRange.FullRange) ?
    AutoFocusRange.FullRange :
    focusControl.SupportedFocusRanges.FirstOrDefault();
    var focusMode =
    focusControl.SupportedFocusModes.Contains(FocusMode.Single) ?
    FocusMode.Single : focusControl.SupportedFocusModes.FirstOrDefault();
    var settings = new FocusSettings { Mode = focusMode, AutoFocusRange =
    focusRange };
    focusControl.Configure(settings);

    var roiControl =
    _mediaCapture.VideoDeviceController.RegionsOfInterestControl;
    await roiControl.SetRegionsAsync(new[] { regionOfInterest }, true);

    await focusControl.FocusAsync();
}

```

In the **TapUnfocus** helper method, obtain the **RegionsOfInterestControl** and call **ClearRegionsAsync** to clear the region that was registered with the control within the **TapToFocus** helper method. Then, get the **FocusControl** and call **FocusAsync** to cause the device to refocus without a region of interest.

C#

```

private async Task TapUnfocus()
{
    _isFocused = false;

    var roiControl =
    _mediaCapture.VideoDeviceController.RegionsOfInterestControl;
    await roiControl.ClearRegionsAsync();

    var focusControl = _mediaCapture.VideoDeviceController.FocusControl;

```

```
        await focusControl.FocusAsync();
    }
```

The **GetPreviewStreamRectInControl** helper method uses the resolution of the preview stream and the orientation of the device to determine the rectangle within the preview element that contains the preview stream, trimming off any letterboxed padding that the control may provide to maintain the stream's aspect ratio. This method uses class member variables defined in the basic media capture example code found in [Basic photo, video, and audio capture with MediaCapture](#).

C#

```
public Rect GetPreviewStreamRectInControl()
{
    var result = new Rect();

    var previewResolution =
        _mediaCapture.VideoDeviceController.GetMediaStreamProperties(MediaStreamType
            .VideoPreview) as VideoEncodingProperties;

    // In case this function is called before everything is initialized
    // correctly, return an empty result
    if (PreviewControl == null || PreviewControl.ActualHeight < 1 ||
        PreviewControl.ActualWidth < 1 ||
        previewResolution == null || previewResolution.Height == 0 ||
        previewResolution.Width == 0)
    {
        return result;
    }

    var streamWidth = previewResolution.Width;
    var streamHeight = previewResolution.Height;

    // For portrait orientations, the width and height need to be swapped
    if (_displayOrientation == DisplayOrientations.Portrait ||
        _displayOrientation == DisplayOrientations.PortraitFlipped)
    {
        streamWidth = previewResolution.Height;
        streamHeight = previewResolution.Width;
    }

    // Start by assuming the preview display area in the control spans the
    // entire width and height both (this is corrected in the next if for the
    // necessary dimension)
    result.Width = PreviewControl.ActualWidth;
    result.Height = PreviewControl.ActualHeight;

    // If UI is "wider" than preview, letterboxing will be on the sides
    if ((PreviewControl.ActualWidth / PreviewControl.ActualHeight >
        streamWidth / (double)streamHeight))
    {
        var scale = PreviewControl.ActualHeight / streamHeight;
```

```

        var scaledWidth = streamWidth * scale;

        result.X = (PreviewControl.ActualWidth - scaledWidth) / 2.0;
        result.Width = scaledWidth;
    }
    else // Preview stream is "wider" than UI, so letterboxing will be on
the top+bottom
    {
        var scale = PreviewControl.ActualWidth / streamWidth;
        var scaledHeight = streamHeight * scale;

        result.Y = (PreviewControl.ActualHeight - scaledHeight) / 2.0;
        result.Height = scaledHeight;
    }

    return result;
}

```

The **ConvertUiTapToPreviewRect** helper method takes as arguments the location of the tap event, the desired size of the focus region, and the rectangle containing the preview stream obtained from the **GetPreviewStreamRectInControl** helper method. This method uses these values and the device's current orientation to calculate the rectangle within the preview stream that contains the desired region. Once again, this method uses class member variables defined in the basic media capture example code found in [Capture Photos and Video with MediaCapture](#).

C#

```

private Rect ConvertUiTapToPreviewRect(Point tap, Size size, Rect
previewRect)
{
    // Adjust for the resulting focus rectangle to be centered around the
position
    double left = tap.X - size.Width / 2, top = tap.Y - size.Height / 2;

    // Get the information about the active preview area within the
CaptureElement (in case it's letterboxed)
    double previewWidth = previewRect.Width, previewHeight =
previewRect.Height;
    double previewLeft = previewRect.Left, previewTop = previewRect.Top;

    // Transform the left and top of the tap to account for rotation
    switch (_displayOrientation)
    {
        case DisplayOrientations.Portrait:
            var tempLeft = left;

            left = top;
            top = previewRect.Width - tempLeft;
            break;
        case DisplayOrientations.LandscapeFlipped:

```

```

        left = previewRect.Width - left;
        top = previewRect.Height - top;
        break;
    case DisplayOrientations.PortraitFlipped:
        var tempTop = top;

        top = left;
        left = previewRect.Width - tempTop;
        break;
    }

    // For portrait orientations, the information about the active preview
    area needs to be rotated
    if (_displayOrientation == DisplayOrientations.Portrait ||
        _displayOrientation == DisplayOrientations.PortraitFlipped)
    {
        previewWidth = previewRect.Height;
        previewHeight = previewRect.Width;
        previewLeft = previewRect.Top;
        previewTop = previewRect.Left;
    }

    // Normalize width and height of the focus rectangle
    var width = size.Width / previewWidth;
    var height = size.Height / previewHeight;

    // Shift rect left and top to be relative to just the active preview
    area
    left -= previewLeft;
    top -= previewTop;

    // Normalize left and top
    left /= previewWidth;
    top /= previewHeight;

    // Ensure rectangle is fully contained within the active preview area
    horizontally
    left = Math.Max(left, 0);
    left = Math.Min(1 - width, left);

    // Ensure rectangle is fully contained within the active preview area
    vertically
    top = Math.Max(top, 0);
    top = Math.Min(1 - height, top);

    // Create and return resulting rectangle
    return new Rect(left, top, width, height);
}

```

Manual focus

The manual focus technique uses a **Slider** control to set the current focus depth of the capture device. A radio button is used to toggle manual focus on and off.

XML

```
<Slider Name="FocusSlider" IsEnabled="{Binding  
ElementName=ManualFocusRadioButton,Path=IsChecked}"  
ValueChanged="FocusSlider_ValueChanged"/>  
<TextBlock Text="{Binding  
ElementName=FocusSlider,Path=Value,FallbackValue='0'}"/>  
<RadioButton Content="Manual" Name="ManualFocusRadioButton"  
Checked="ManualFocusRadioButton_Checked" IsChecked="False"/>
```

Check to see if the current capture device supports the **FocusControl** by checking the **Supported** property. If the control is supported, you can show and enable the UI for this feature.

The focus value must be within the range supported by the device and must be an increment of the supported step size. Get the supported values for the current device by checking the **Min**, **Max**, and **Step** properties, which are used to set the corresponding properties of the slider control.

Set the slider control's value to the current value of the **FocusControl** after unregistering the **ValueChanged** event handler so that the event is not triggered when the value is set.

C#

```
var focusControl = _mediaCapture.VideoDeviceController.FocusControl;  
  
if (focusControl.Supported)  
{  
    FocusSlider.Visibility = Visibility.Visible;  
    ManualFocusRadioButton.Visibility = Visibility.Visible;  
  
    FocusSlider.Minimum = focusControl.Min;  
    FocusSlider.Maximum = focusControl.Max;  
    FocusSlider.StepFrequency = focusControl.Step;  
  
    FocusSlider.ValueChanged -= FocusSlider_ValueChanged;  
    FocusSlider.Value = focusControl.Value;  
    FocusSlider.ValueChanged += FocusSlider_ValueChanged;  
}  
else  
{  
    FocusSlider.Visibility = Visibility.Collapsed;  
    ManualFocusRadioButton.Visibility = Visibility.Collapsed;  
}
```

In the **Checked** event handler for the manual focus radio button, get the **FocusControl** object and call **LockAsync** in case your app had previously unlocked the focus with a call to **UnlockAsync**.

C#

```
private async void ManualFocusRadioButton_Checked(object sender,
RoutedEventArgs e)
{
    var focusControl = _mediaCapture.VideoDeviceController.FocusControl;
    await focusControl.LockAsync();
}
```

In the **ValueChanged** event handler of the manual focus slider, get the current value of the control and then set the focus value by calling **SetValueAsync**.

C#

```
private async void FocusSlider_ValueChanged(object sender,
Windows.UI.Xaml.Controls.Primitives.RangeBaseValueChangedEventArgs e)
{
    var value = (sender as Slider).Value;
    await
_mediaCapture.VideoDeviceController.FocusControl.SetValueAsync((uint)value);
}
```

Enable the focus light

On devices that support it, you can enable a focus assist light to help the device focus. This example uses a checkbox to enable or disable the focus assist light.

XML

```
<CheckBox Content="Assist Light" Name="FocusLightCheckBox" IsEnabled="
{Binding ElementName=TapFocusRadioButton,Path=IsChecked}"
          Checked="FocusLightCheckBox_CheckedChanged"
          Unchecked="FocusLightCheckBox_CheckedChanged"/>
```

Check to see if the current capture device supports the **FlashControl** by checking the **Supported** property. Also check the **AssistantLightSupported** to make sure the assist light is also supported. If these are both supported, you can show and enable the UI for this feature.

C#

```

var focusControl = _mediaCapture.VideoDeviceController.FocusControl;

if (focusControl.Supported)
{
    FocusLightCheckBox.Visibility =
        (_mediaCapture.VideoDeviceController.FlashControl.Supported &&
        _mediaCapture.VideoDeviceController.FlashControl.AssistantLightSupported) ?
        Visibility.Visible : Visibility.Collapsed;
}
else
{
    FocusLightCheckBox.Visibility = Visibility.Collapsed;
}

```

In the **CheckedChanged** event handler, get the capture devices **FlashControl** object. Set the **AssistantLightEnabled** property to enable or disable the focus light.

C#

```

private void FocusLightCheckBox_CheckedChanged(object sender,
RoutedEventArgs e)
{
    var flashControl = _mediaCapture.VideoDeviceController.FlashControl;

    flashControl.AssistantLightEnabled = (FocusLightCheckBox.IsChecked ==
true);
}

```

ISO speed

The **IsoSpeedControl** allows you to set the ISO speed used during photo or video capture.

This example uses a **Slider** control to adjust the current exposure compensation value and a checkbox to toggle automatic ISO speed adjustment.

XML

```

<Slider Name="IsoSlider" ValueChanged="IsoSlider_ValueChanged"/>
<TextBlock Text="{Binding ElementName=IsoSlider,Path=Value}" Visibility="
{Binding ElementName=IsoSlider,Path=Visibility}"/>
<CheckBox Name="IsoAutoCheckBox" Content="Auto"
Checked="IsoAutoCheckBox_CheckedChanged"
Unchecked="IsoAutoCheckBox_CheckedChanged"/>

```


Check to see if the current capture device supports the **IsoSpeedControl** by checking the **Supported** property. If the control is supported, you can show and enable the UI for this feature. Set the checked state of the checkbox to indicate if automatic ISO speed adjustment is currently active to the value of the **Auto** property.

The ISO speed value must be within the range supported by the device and must be an increment of the supported step size. Get the supported values for the current device by checking the **Min**, **Max**, and **Step** properties, which are used to set the corresponding properties of the slider control.

Set the slider control's value to the current value of the **IsoSpeedControl** after unregistering the **ValueChanged** event handler so that the event is not triggered when the value is set.

C#

```
private void UpdateIsoControlCapabilities()
{
    var isoSpeedControl =
        _mediaCapture.VideoDeviceController.IsoSpeedControl;

    if (isoSpeedControl.Supported)
    {
        IsoAutoCheckBox.Visibility = Visibility.Visible;
        IsoSlider.Visibility = Visibility.Visible;

        IsoAutoCheckBox.IsChecked = isoSpeedControl.Auto;

        IsoSlider.Minimum = isoSpeedControl.Min;
        IsoSlider.Maximum = isoSpeedControl.Max;
        IsoSlider.StepFrequency = isoSpeedControl.Step;

        IsoSlider.ValueChanged -= IsoSlider_ValueChanged;
        IsoSlider.Value = isoSpeedControl.Value;
        IsoSlider.ValueChanged += IsoSlider_ValueChanged;
    }
    else
    {
        IsoAutoCheckBox.Visibility = Visibility.Collapsed;
        IsoSlider.Visibility = Visibility.Collapsed;
    }
}
```

In the **ValueChanged** event handler, get the current value of the control and then set the ISO speed value by calling **SetValueAsync**.

C#

```
private async void IsoSlider_ValueChanged(object sender,
Windows.UI.Xaml.Controls.Primitives.RangeBaseValueChangedEventArgs e)
{
    var value = (sender as Slider).Value;
    await
_mediaCapture.VideoDeviceController.IsoSpeedControl.SetValueAsync((uint)valu
e);
}
```

In the **CheckedChanged** event handler of the auto ISO speed checkbox, turn on automatic ISO speed adjustment by calling **SetAutoAsync**. Turn automatic ISO speed adjustment off by calling **SetValueAsync** and passing in the current value of the slider control.

C#

```
private async void IsoAutoCheckBox_CheckedChanged(object sender,
RoutedEventArgs e)
{
    var autoIso = (sender as CheckBox).IsChecked == true;

    if (autoIso)
    {
        await
_mediaCapture.VideoDeviceController.IsoSpeedControl.SetAutoAsync();
    }
    else
    {
        await
_mediaCapture.VideoDeviceController.IsoSpeedControl.SetValueAsync((uint)IsoS
lider.Value);
    }
}
```

Optical image stabilization

Optical image stabilization (OIS) stabilizes a captured video stream by mechanically manipulating the hardware capture device, which can provide a superior result than digital stabilization. On devices that don't support OIS, you can use the **VideoStabilizationEffect** to perform digital stabilization on your captured video. For more information, see [Effects for video capture](#).

Determine if OIS is supported on the current device by checking the **OpticalImageStabilizationControl.Supported** property.

The OIS control supports three modes: on, off, and automatic, which means that the device dynamically determines if OIS would improve the media capture and, if so, enables OIS. To determine if a particular mode is supported on a device, check to see if the [OpticalImageStabilizationControl.SupportedModes](#) collection contains the desired mode.

Enable or disable OIS by setting the [OpticalImageStabilizationControl.Mode](#) to the desired mode.

C#

```
private void SetOpticalImageStabilizationMode(OpticalImageStabilizationMode
mode)
{
    if
(!_mediaCapture.VideoDeviceController.OpticalImageStabilizationControl.Supp
orted)
    {
        ShowMessageToUser("Optical image stabilization not available");
        return;
    }

    var stabilizationModes =
_mediaCapture.VideoDeviceController.OpticalImageStabilizationControl.Support
edModes;

    if (!stabilizationModes.Contains(mode))
    {
        ShowMessageToUser("Optical image stabilization setting not
supported");
        return;
    }

    _mediaCapture.VideoDeviceController.OpticalImageStabilizationControl.Mode =
mode;
}
```

Powerline frequency

Some camera devices support anti-flicker processing that depends on knowing the AC frequency of the powerlines in the current environment. Some devices support automatic determination of the powerline frequency, while others require that the frequency be set manually. The following code example shows how to determine powerline frequency support on the device and, if needed, how to set the frequency manually.

First, call the **VideoDeviceController** method **TryGetPowerlineFrequency**, passing in an output parameter of type **PowerlineFrequency**; if this call fails, the powerline frequency control is not supported on the current device. If the feature is supported, you can determine if automatic mode is available on the device by trying to set auto mode. Do this by calling **TrySetPowerlineFrequency** and passing in the value **Auto**. If the call succeeds, that means that your auto powerline frequency is supported. If the powerline frequency controller is supported on the device but automatic frequency detection is not, you can still manually set the frequency by using **TrySetPowerlineFrequency**. In this example, **MyCustomFrequencyLookup** is a custom method that you implement to determine the correct frequency for the device's current location.

C#

```
PowerlineFrequency getFrequency;

if (! _mediaCapture.VideoDeviceController.TryGetPowerlineFrequency(out
getFrequency))
{
    // Powerline frequency is not supported on this device.
    return;
}

if (!
_mediaCapture.VideoDeviceController.TrySetPowerlineFrequency(PowerlineFreque
ncy.Auto))
{
    // Set the frequency manually
    PowerlineFrequency setFrequency = MyCustomFrequencyLookup();
    if
(_mediaCapture.VideoDeviceController.TrySetPowerlineFrequency(setFrequency))
    {
        System.Diagnostics.Debug.WriteLine(String.Format("Powerline
frequency manually set to {0}.", setFrequency));
    }
}
```

White balance

The **WhiteBalanceControl** allows you to set the white balance used during photo or video capture.

This example uses a **ComboBox** control to select from built-in color temperature presets and a **Slider** control for manual white balance adjustment.

XML

```

<Slider Name="WbSlider" ValueChanged="WbSlider_ValueChanged"/>
<TextBlock Name="WbTextBox" Text="{Binding ElementName=WbSlider,Path=Value}"
Visibility="{Binding ElementName=WbSlider,Path=Visibility}"/>
<ComboBox Name="WbComboBox" SelectionChanged="WbComboBox_SelectionChanged"/>

```

Check to see if the current capture device supports the **WhiteBalanceControl** by checking the **Supported** property. If the control is supported, you can show and enable the UI for this feature. Set the items of the combo box to the values of the **ColorTemperaturePreset** enumeration. And set the selected item to the current value of the **Preset** property.

For manual control, the white balance value must be within the range supported by the device and must be an increment of the supported step size. Get the supported values for the current device by checking the **Min**, **Max**, and **Step** properties, which are used to set the corresponding properties of the slider control. Before enabling manual control, check to make sure that the range between the minimum and maximum supported values is greater than the step size. If it is not, manual control is not supported on the current device.

Set the slider control's value to the current value of the **WhiteBalanceControl** after unregistering the **ValueChanged** event handler so that the event is not triggered when the value is set.

C#

```

var whiteBalanceControl =
_mediaCapture.VideoDeviceController.WhiteBalanceControl;

if (whiteBalanceControl.Supported)
{
    WbSlider.Visibility = Visibility.Visible;
    WbComboBox.Visibility = Visibility.Visible;

    if (WbComboBox.ItemsSource == null)
    {
        WbComboBox.ItemsSource =
Enum.GetValues(typeof(ColorTemperaturePreset)).Cast<ColorTemperaturePreset>
();
    }

    WbComboBox.SelectedItem = whiteBalanceControl.Preset;

    if (whiteBalanceControl.Max - whiteBalanceControl.Min >
whiteBalanceControl.Step)
    {

        WbSlider.Minimum = whiteBalanceControl.Min;
        WbSlider.Maximum = whiteBalanceControl.Max;
    }
}

```

```

        WbSlider.StepFrequency = whiteBalanceControl.Step;

        WbSlider.ValueChanged -= WbSlider_ValueChanged;
        WbSlider.Value = whiteBalanceControl.Value;
        WbSlider.ValueChanged += WbSlider_ValueChanged;
    }
    else
    {
        WbSlider.Visibility = Visibility.Collapsed;
    }
}
else
{
    WbSlider.Visibility = Visibility.Collapsed;
    WbComboBox.Visibility = Visibility.Collapsed;
}

```

In the [SelectionChanged](#) event handler of the color temperature preset combo box, get the currently selected preset and set the value of the control by calling [SetPresetAsync](#). If the selected preset value is not **Manual**, disable the manual white balance slider.

C#

```

private async void WbComboBox_SelectionChanged(object sender,
SelectionChangedEventArgs e)
{
    if(!_isPreviewing)
    {
        // Do not set white balance values unless the preview stream is
        running.
        return;
    }

    var selected = (ColorTemperaturePreset)WbComboBox.SelectedItem;
    WbSlider.IsEnabled = (selected == ColorTemperaturePreset.Manual);
    await
_mediaCapture.VideoDeviceController.WhiteBalanceControl.SetPresetAsync(selected);
}

```

In the **ValueChanged** event handler, get the current value of the control and the set the white balance value by calling [SetValueAsync](#).

C#

```

private async void WbSlider_ValueChanged(object sender,
Windows.UI.Xaml.Controls.Primitives.RangeBaseValueChangedEventArgs e)
{
    if (!_isPreviewing)
    {

```

```

        // Do not set white balance values unless the preview stream is
        running.
        return;
    }

    var value = (sender as Slider).Value;
    await
_mediaCapture.VideoDeviceController.WhiteBalanceControl.SetValueAsync((uint)
value);
}

```

❗ Important

Adjusting the white balance is only supported while the preview stream is running. Check to make sure that the preview stream is running before setting the white balance value or preset.

❗ Important

The **ColorTemperaturePreset.Auto** preset value instructs the system to automatically adjust the white balance level. For some scenarios, such as capturing a photo sequence where the white balance levels should be the same for each frame, you may want to lock the control to the current automatic value. To do this, call [SetPresetAsync](#) and specify the **Manual** preset and do not set a value on the control using [SetValueAsync](#). This will cause the device to lock the current value. Do not attempt to read the current control value and then pass the returned value into [SetValueAsync](#) because this value is not guaranteed to be correct.

Zoom

The [ZoomControl](#) allows you to set the zoom level used during photo or video capture.

This example uses a [Slider](#) control to adjust the current zoom level. The following section shows how to adjust zoom based on a pinch gesture on the screen.

XML

```

<Slider Name="ZoomSlider" Grid.Row="0" Orientation="Vertical"
HorizontalAlignment="Center" VerticalAlignment="Stretch"
ValueChanged="ZoomSlider_ValueChanged"/>
<TextBlock Grid.Row="1" HorizontalAlignment="Center" Text="{Binding
ElementName=ZoomSlider,Path=Value}"/>

```

Check to see if the current capture device supports the **ZoomControl** by checking the **Supported** property. If the control is supported, you can show and enable the UI for this feature.

The zoom level value must be within the range supported by the device and must be an increment of the supported step size. Get the supported values for the current device by checking the **Min**, **Max**, and **Step** properties, which are used to set the corresponding properties of the slider control.

Set the slider control's value to the current value of the **ZoomControl** after unregistering the **ValueChanged** event handler so that the event is not triggered when the value is set.

C#

```
var zoomControl = _mediaCapture.VideoDeviceController.ZoomControl;

if (zoomControl.Supported)
{
    ZoomSlider.Visibility = Visibility.Visible;

    ZoomSlider.Minimum = zoomControl.Min;
    ZoomSlider.Maximum = zoomControl.Max;
    ZoomSlider.StepFrequency = zoomControl.Step;

    ZoomSlider.ValueChanged -= ZoomSlider_ValueChanged;
    ZoomSlider.Value = zoomControl.Value;
    ZoomSlider.ValueChanged += ZoomSlider_ValueChanged;
}
else
{
    ZoomSlider.Visibility = Visibility.Collapsed;
}
```

In the **ValueChanged** event handler, create a new instance of the **ZoomSettings** class, setting the **Value** property to the current value of the zoom slider control. If the **SupportedModes** property of the **ZoomControl** contains **ZoomTransitionMode.Smooth**, it means the device supports smooth transitions between zoom levels. Since this modes provides a better user experience, you will typically want to use this value for the **Mode** property of the **ZoomSettings** object.

Finally, change the current zoom settings by passing your **ZoomSettings** object into the **Configure** method of the **ZoomControl** object.

C#

```
private void ZoomSlider_ValueChanged(object sender,
Windows.UI.Xaml.Controls.Primitives.RangeBaseValueChangedEventArgs e)
{
```



```

var level = (float)ZoomSlider.Value;
var settings = new ZoomSettings { Value = level };

var zoomControl = _mediaCapture.VideoDeviceController.ZoomControl;
if (zoomControl.SupportedModes.Contains(ZoomTransitionMode.Smooth))
{
    settings.Mode = ZoomTransitionMode.Smooth;
}
else
{
    settings.Mode = zoomControl.SupportedModes.First();
}

zoomControl.Configure(settings);
}

```

Smooth zoom using pinch gesture

As discussed in the previous section, on devices that support it, smooth zoom mode allows the capture device to smoothly transition between digital zoom levels, allowing the user to dynamically adjust the zoom level during the capture operation without discrete and jarring transitions. This section describes how to adjust the zoom level in response to a pinch gesture.

First, determine if the digital zoom control is supported on the current device by checking the [ZoomControl.Supported](#) property. Next, determine if smooth zoom mode is available by checking the [ZoomControl.SupportedModes](#) to see if it contains the value [ZoomTransitionMode.Smooth](#).

C#

```

private bool IsSmoothZoomSupported()
{
    if (!_mediaCapture.VideoDeviceController.ZoomControl.Supported)
    {
        ShowMessageToUser("Digital zoom is not supported on this device.");
        return false;
    }

    var zoomModes =
        _mediaCapture.VideoDeviceController.ZoomControl.SupportedModes;

    if (!zoomModes.Contains(ZoomTransitionMode.Smooth))
    {
        ShowMessageToUser("Smooth zoom not supported");
        return false;
    }
}

```

```
        return true;
    }
```

On a multi-touch enabled device, a typical scenario is to adjust the zoom factor based on a two-finger pinch gesture. Set the [ManipulationMode](#) property of the [CaptureElement](#) control to [ManipulationModes.Scale](#) to enable the pinch gesture. Then, register for the [ManipulationDelta](#) event which is raised when the pinch gesture changes size.

C#

```
private void RegisterPinchGestureHandler()
{
    if (!IsSmoothZoomSupported())
    {
        return;
    }

    // Enable pinch/zoom gesture for the preview control
    PreviewControl.ManipulationMode = ManipulationModes.Scale;
    PreviewControl.ManipulationDelta += PreviewControl_ManipulationDelta;
}
```

In the handler for the [ManipulationDelta](#) event, update the zoom factor based on the change in the user's pinch gesture. The [ManipulationDelta.Scale](#) value represents the change in scale of the pinch gesture such that a small increase in the size of the pinch is a number slightly larger than 1.0 and a small decrease in the pinch size is a number slightly smaller than 1.0. In this example, the current value of the zoom control is multiplied by the scale delta.

Before setting the zoom factor, you must make sure that the value is not less than the minimum value supported by the device as indicated by the [ZoomControl.Min](#) property. Also, make sure that the value is less than or equal to the [ZoomControl.Max](#) value. Finally, you must make sure that the zoom factor is a multiple of the zoom step size supported by the device as indicated by the [Step](#) property. If your zoom factor does not meet these requirements, an exception will be thrown when you attempt to set the zoom level on the capture device.

Set the zoom level on the capture device by creating a new [ZoomSettings](#) object. Set the [Mode](#) property to [ZoomTransitionMode.Smooth](#) and then set the [Value](#) property to your desired zoom factor. Finally, call [ZoomControl.Configure](#) to set the new zoom value on the device. The device will smoothly transition to the new zoom value.

C#

```
private void PreviewControl_ManipulationDelta(object sender,
ManipulationDeltaRoutedEventArgs e)
{
    var zoomControl = _mediaCapture.VideoDeviceController.ZoomControl;

    // Example zoom factor calculation based on size of scale gesture
    var zoomFactor = zoomControl.Value * e.Delta.Scale;

    if (zoomFactor < zoomControl.Min) zoomFactor = zoomControl.Min;
    if (zoomFactor > zoomControl.Max) zoomFactor = zoomControl.Max;
    zoomFactor = zoomFactor - (zoomFactor % zoomControl.Step);

    var settings = new ZoomSettings();
    settings.Mode = ZoomTransitionMode.Smooth;
    settings.Value = zoomFactor;

    _mediaCapture.VideoDeviceController.ZoomControl.Configure(settings);
}
```

Related topics

- [Camera](#)
- [Basic photo, video, and audio capture with MediaCapture](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Manual camera controls for video capture

Article • 10/20/2022

This article shows you how to use manual device controls to enable enhanced video capture scenarios, including HDR video and exposure priority.

The video device controls discussed in this article are all added to your app by using the same pattern. First, check to see if the control is supported on the current device on which your app is running. If the control is supported, set the desired mode for the control. Typically, if a particular control is unsupported on the current device, you should disable or hide the UI element that allows the user to enable the feature.

All of the device control APIs discussed in this article are members of the [Windows.Media.Devices](#) namespace.

C#

```
using Windows.Media.Devices;
```

ⓘ Note

This article builds on concepts and code discussed in [Basic photo, video, and audio capture with MediaCapture](#), which describes the steps for implementing basic photo and video capture. We recommend that you familiarize yourself with the basic media capture pattern in that article before moving on to more advanced capture scenarios. The code in this article assumes that your app already has an instance of MediaCapture that has been properly initialized.

HDR video

The high dynamic range (HDR) video feature applies HDR processing to the video stream of the capture device. Determine if HDR video is supported by selecting the [HdrVideoControl.Supported](#) property.

The HDR video control supports three modes: on, off, and automatic, which means that the device dynamically determines if HDR video processing would improve the media capture and, if so, enables HDR video. To determine if a particular mode is supported on

the current device, check to see if the [HdrVideoControl.SupportedModes](#) collection contains the desired mode.

Enable or disable HDR video processing by setting the [HdrVideoControl.Mode](#) to the desired mode. This control requires that the stream is at a stopped state before the mode is set, see [KSPROPERTY_CAMERACONTROL_EXTENDED_VIDEOHDR](#).

C#

```
private void SetHdrVideoMode(HdrVideoMode mode)
{
    if (!_mediaCapture.VideoDeviceController.HdrVideoControl.Supported)
    {
        ShowMessageToUser("HDR Video not available");
        return;
    }

    var hdrVideoModes =
        _mediaCapture.VideoDeviceController.HdrVideoControl.SupportedModes;

    if (!hdrVideoModes.Contains(mode))
    {
        ShowMessageToUser("HDR Video setting not supported");
        return;
    }

    _mediaCapture.VideoDeviceController.HdrVideoControl.Mode = mode;
}
```

Exposure priority

The [ExposurePriorityVideoControl](#), when enabled, evaluates the video frames from the capture device to determine if the video is capturing a low-light scene. If so, the control lowers the frame rate of the captured video in order to increase the exposure time for each frame and improve the visual quality of the captured video.

Determine if the exposure priority control is supported on the current device by checking the [ExposurePriorityVideoControl.Supported](#) property.

Enable or disable the exposure priority control by setting the [ExposurePriorityVideoControl.Enabled](#) to the desired mode.

C#

```
if
(!_mediaCapture.VideoDeviceController.ExposurePriorityVideoControl.Supported
)
```

```
{
    ShowMessageToUser("Exposure priority not available");
    return;
}
_mediaCapture.VideoDeviceController.ExposurePriorityVideoControl.Enabled =
true;
```

Temporal denoising

Starting with Windows 10, version 1803, you can enable temporal denoising for video on devices that support it. This feature fuses the image data from multiple adjacent frames in real time to produce video frames that have less visual noise.

The [VideoTemporalDenoisingControl](#) allows your app to determine if temporal denoising is supported on the current device, and if so, which denoising modes are supported. The available denoising modes are **Off**, **On**, and **Auto**. A device may not support all modes, but every device must support either **Auto** or **On** and **Off**.

The following example uses a simple UI to provide radio buttons allowing the user to switch between denoising modes.

XML

```
<StackPanel Orientation="Vertical" HorizontalAlignment="Right" >
    <StackPanel x:Name="denoiseControls" Visibility="Collapsed">
        <TextBlock>Temporal Denoising</TextBlock>
        <RadioButton x:Name="denoiseOffButton"
Checked="denoiseButton_Checked"
GroupName="Denoise Group" Content="Off"/>
        <RadioButton x:Name="denoiseOnButton"
Checked="denoiseButton_Checked"
GroupName="Denoise Group" Content="On" Visibility="Collapsed"/>
        <RadioButton x:Name="denoiseAutoButton"
Checked="denoiseButton_Checked"
GroupName="Denoise Group" Content="Auto" Visibility="Collapsed"/>
    </StackPanel>
</StackPanel>
```

In the following method, the [VideoTemporalDenoisingControl.Supported](#) property is checked to see if temporal denoising is supported at all on the current device. If so, then we check to make sure that **Off** and **Auto** or **On** is supported, in which case we make our radio buttons visible. Next, the **Auto** and **On** buttons are made visible if those methods are supported.

C#

```

private void UpdateDenoiseCapabilities()
{
    if
    (_mediaCapture.VideoDeviceController.VideoTemporalDenoisingControl.Supported
    )
    {
        IReadOnlyList<VideoTemporalDenoisingMode> modes =
        _mediaCapture.VideoDeviceController.VideoTemporalDenoisingControl.SupportedM
        odes;

        if(modes.Contains(Windows.Media.Devices.VideoTemporalDenoisingMode.Off) &&
        (modes.Contains(Windows.Media.Devices.VideoTemporalDenoisingMode.On) ||
        modes.Contains(Windows.Media.Devices.VideoTemporalDenoisingMode.Auto)))
        {
            denoiseControls.Visibility = Visibility.Visible;

            if
            (modes.Contains(Windows.Media.Devices.VideoTemporalDenoisingMode.On))
            {
                denoiseOnButton.Visibility = Visibility.Visible;
            }
            if
            (modes.Contains(Windows.Media.Devices.VideoTemporalDenoisingMode.Auto))
            {
                denoiseAutoButton.Visibility = Visibility.Visible;
            }
        }
    }
}

```

In the **Checked** event handler for the radio buttons, the name of the button is checked and the corresponding mode is set by setting the **VideoTemporalDenoisingControl.Mode** property.

C#

```

private void denoiseButton_Checked(object sender, RoutedEventArgs e)
{
    var button = sender as RadioButton;
    if(button.Name == "denoiseOffButton")
    {
        _mediaCapture.VideoDeviceController.VideoTemporalDenoisingControl.Mode =
        Windows.Media.Devices.VideoTemporalDenoisingMode.Off;
    }
    else if (button.Name == "denoiseOnButton")
    {
        _mediaCapture.VideoDeviceController.VideoTemporalDenoisingControl.Mode =
        Windows.Media.Devices.VideoTemporalDenoisingMode.On;
    }
}

```

```

    }
    else if (button.Name == "denoiseAutoButton")
    {

        _mediaCapture.VideoDeviceController.VideoTemporalDenoisingControl.Mode =
        Windows.Media.Devices.VideoTemporalDenoisingMode.Auto;
    }
}

```

Disabling temporal denoising while processing frames

Video that has been processed using temporal denoising can be more pleasing to the human eye. However, because temporal denoising can impact image consistency and decrease the amount of details in the frame, apps that perform image processing on the frames, such as registration or optical character recognition, may want to programmatically disable denoising when image processing is enabled.

The following example determines which denoising modes are supported and stores this information in some class variables.

C#

```

private bool _isVideoTemporalDenoisingOffSupported = false;
private bool _isProcessing = false;
private Windows.Media.Devices.VideoTemporalDenoisingMode?
_videoDenoisingEnabledMode = null;

```

C#

```

private void ConfigureDenoiseForFrameProcessing()
{
    if
    (_mediaCapture.VideoDeviceController.VideoTemporalDenoisingControl.Supported
    )
    {
        // Query support for the VideoTemporalDenoising control Off mode
        _isVideoTemporalDenoisingOffSupported =
        _mediaCapture.VideoDeviceController.VideoTemporalDenoisingControl.SupportedM
odes.Contains(Windows.Media.Devices.VideoTemporalDenoisingMode.Off);

        // Query support for a mode that would enable VideoTemporalDenoising
        (On or Auto) and toggle it if supported
        if
        (_mediaCapture.VideoDeviceController.VideoTemporalDenoisingControl.Supported
        Modes.Contains(Windows.Media.Devices.VideoTemporalDenoisingMode.On))
        {
            _videoDenoisingEnabledMode =
            Windows.Media.Devices.VideoTemporalDenoisingMode.On;

```



```

_mediaCapture.VideoDeviceController.VideoTemporalDenoisingControl.Mode =
(Windows.Media.Devices.VideoTemporalDenoisingMode)_videoDenoisingEnabledMode
;
    }
    else if
(_mediaCapture.VideoDeviceController.VideoTemporalDenoisingControl.Supported
Modes.Contains(Windows.Media.Devices.VideoTemporalDenoisingMode.Auto))
    {
        _videoDenoisingEnabledMode =
Windows.Media.Devices.VideoTemporalDenoisingMode.Auto;

_mediaCapture.VideoDeviceController.VideoTemporalDenoisingControl.Mode =
(Windows.Media.Devices.VideoTemporalDenoisingMode)_videoDenoisingEnabledMode
;
    }
}
}

```

When the app enables frame processing, it sets the denoising mode to **Off** if that mode is supported so that the frame processing can use raw frames that have not been denoised.

C#

```

public void EnableFrameProcessing()
{
    // Toggle Off VideoTemporalDenoising
    if (_isVideoTemporalDenoisingOffSupported)
    {

_mediaCapture.VideoDeviceController.VideoTemporalDenoisingControl.Mode =
Windows.Media.Devices.VideoTemporalDenoisingMode.Off;
    }

    _isProcessing = true;
}

```

When the app disables frame processing, it sets the denoising mode to **On** or **Auto**, depending on which mode is supported.

C#

```

public void DisableFrameProcessing()
{
    _isProcessing = false;

    // If a VideoTemporalDenoising mode to enable VideoTemporalDenoising is
supported, toggle it
    if (_videoDenoisingEnabledMode != null)
    {

```

```
_mediaCapture.VideoDeviceController.VideoTemporalDenoisingControl.Mode =  
(Windows.Media.Devices.VideoTemporalDenoisingMode)_videoDenoisingEnabledMode  
;  
    }  
}
```

For more information on obtaining video frames for image processing, see [Process media frames with MediaFrameReader](#).

Related topics

- [Camera](#)
- [Basic photo, video, and audio capture with MediaCapture](#)
- [Process media frames with MediaFrameReader](#)
- [VideoTemporalDenoisingControl](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Effects for video capture

Article • 10/20/2022

This topic shows you how to apply effects to the camera preview and recording video streams and shows you how to use the video stabilization effect.

ⓘ Note

This article builds on concepts and code discussed in [Basic photo, video, and audio capture with MediaCapture](#), which describes the steps for implementing basic photo and video capture. We recommend that you familiarize yourself with the basic media capture pattern in that article before moving on to more advanced capture scenarios. The code in this article assumes that your app already has an instance of `MediaCapture` that has been properly initialized.

Adding and removing effects from the camera video stream

To capture or preview video from the device's camera, you use the [MediaCapture](#) object as described in [Basic photo, video, and audio capture with MediaCapture](#). After you have initialized the `MediaCapture` object, you can add one or more video effects to the preview or capture stream by calling [AddVideoEffectAsync](#), passing in an [IVideoEffectDefinition](#) object representing the effect to be added, and a member of the [MediaStreamType](#) enumeration indicating whether the effect should be added to the camera's preview stream or the record stream.

ⓘ Note

On some devices, the preview stream and the capture stream are the same, which means that if you specify `MediaStreamType.VideoPreview` or `MediaStreamType.VideoRecord` when you call `AddVideoEffectAsync`, the effect will be applied to both preview and record streams. You can determine whether the preview and record streams are the same on the current device by checking the `VideoDeviceCharacteristic` property of the `MediaCaptureSettings` for the `MediaCapture` object. If the value of this property is `VideoDeviceCharacteristic.AllStreamsIdentical` or `VideoDeviceCharacteristic.PreviewRecordStreamsIdentical`, then the streams are the same and any effect you apply to one will affect the other.

The following example adds an effect to both the camera preview and record streams. This example illustrates checking to see if the record and preview streams are the same.

C#

```
if (mediaCapture.MediaCaptureSettings.VideoDeviceCharacteristic ==
    VideoDeviceCharacteristic.AllStreamsIdentical ||
    mediaCapture.MediaCaptureSettings.VideoDeviceCharacteristic ==
    VideoDeviceCharacteristic.PreviewRecordStreamsIdentical)
{
    // This effect will modify both the preview and the record streams,
    // because they are the same stream.
    myRecordEffect = await
mediaCapture.AddVideoEffectAsync(myEffectDefinition,
MediaStreamType.VideoRecord);
}
else
{
    myRecordEffect = await
mediaCapture.AddVideoEffectAsync(myEffectDefinition,
MediaStreamType.VideoRecord);
    myPreviewEffect = await
mediaCapture.AddVideoEffectAsync(myEffectDefinition,
MediaStreamType.VideoPreview);
}
```

Note that **AddVideoEffectAsync** returns an object that implements [IMediaExtension](#) that represents the added video effect. Some effects allow you to change the effect settings by passing a [PropertySet](#) into the [SetProperties](#) method.

Starting with Windows 10, version 1607, you can also use the object returned by **AddVideoEffectAsync** to remove the effect from the video pipeline by passing it into [RemoveEffectAsync](#). **RemoveEffectAsync** automatically determines whether the effect object parameter was added to the preview or record stream, so you don't need to specify the stream type when making the call.

C#

```
if (myRecordEffect != null)
{
    await mediaCapture.RemoveEffectAsync(myRecordEffect);
}
if(myPreviewEffect != null)
{
    await mediaCapture.RemoveEffectAsync(myPreviewEffect);
}
```

You can also remove all effects from the preview or capture stream by calling [ClearEffectsAsync](#) and specifying the stream for which all effects should be removed.

C#

```
await mediaCapture.ClearEffectsAsync(MediaStreamType.VideoPreview);  
await mediaCapture.ClearEffectsAsync(MediaStreamType.VideoRecord);
```

Video stabilization effect

The video stabilization effect manipulates the frames of a video stream to minimize shaking caused by holding the capture device in your hand. Because this technique causes the pixels to be shifted right, left, up, and down, and because the effect can't know what the content just outside the video frame is, the stabilized video is cropped slightly from the original video. A utility function is provided to allow you to adjust your video encoding settings to optimally manage the cropping performed by the effect.

On devices that support it, Optical Image Stabilization (OIS) stabilizes video by mechanically manipulating the capture device and, therefore, does not need to crop the edges of the video frames. For more information, see [Capture device controls for video capture](#).

Set up your app to use video stabilization

In addition to the namespaces required for basic media capture, using the video stabilization effect requires the following namespace.

C#

```
using Windows.Media.Core;  
using Windows.Media.MediaProperties;  
using Windows.Media.Effects;  
using Windows.Media;
```

Declare a member variable to store the [VideoStabilizationEffect](#) object. As part of the effect implementation, you will modify the encoding properties that you use to encode the captured video. Declare two variables to store a backup copy of the initial input and output encoding properties so that you can restore them later when the effect is disabled. Finally, declare a member variable of type [MediaEncodingProfile](#) because this object will be accessed from multiple locations within your code.

C#

```
private VideoStabilizationEffect _videoStabilizationEffect;  
private VideoEncodingProperties _inputPropertiesBackup;  
private VideoEncodingProperties _outputPropertiesBackup;  
private MediaEncodingProfile _encodingProfile;
```

For this scenario, you should assign the media encoding profile object to a member variable so that you can access it later.

C#

```
_encodingProfile =  
MediaEncodingProfile.CreateMp4(VideoEncodingQuality.Auto);
```

Initialize the video stabilization effect

After your **MediaCapture** object has been initialized, create a new instance of the **VideoStabilizationEffectDefinition** object. Call **MediaCapture.AddVideoEffectAsync** to add the effect to the video pipeline and retrieve an instance of the **VideoStabilizationEffect** class. Specify **MediaStreamType.VideoRecord** to indicate that the effect should be applied to the video record stream.

Register an event handler for the **EnabledChanged** event and call the helper method **SetUpVideoStabilizationRecommendationAsync**, both of which are discussed later in this article. Finally, set the **Enabled** property of the effect to true to enable the effect.

C#

```
// Create the effect definition  
VideoStabilizationEffectDefinition stabilizerDefinition = new  
VideoStabilizationEffectDefinition();  
  
// Add the video stabilization effect to media capture  
_videoStabilizationEffect =  
    (VideoStabilizationEffect)await  
mediaCapture.AddVideoEffectAsync(stabilizerDefinition,  
MediaStreamType.VideoRecord);  
  
_videoStabilizationEffect.EnabledChanged +=  
VideoStabilizationEffect_EnabledChanged;  
  
await SetUpVideoStabilizationRecommendationAsync();  
  
_videoStabilizationEffect.Enabled = true;
```

Use recommended encoding properties

As discussed earlier in this article, the technique that the video stabilization effect uses necessarily causes the stabilized video to be cropped slightly from the source video. Define the following helper function in your code in order to adjust the video encoding properties to optimally handle this limitation of the effect. This step is not required in order to use the video stabilization effect, but if you don't perform this step, the resulting video will be upscaled slightly and therefore have slightly lower visual fidelity.

Call [GetRecommendedStreamConfiguration](#) on your video stabilization effect instance, passing in the [VideoDeviceController](#) object, which informs the effect about your current input stream encoding properties, and your **MediaEncodingProfile** which lets the effect know your current output encoding properties. This method returns a [VideoStreamConfiguration](#) object containing new recommended input and output stream encoding properties.

The recommended input encoding properties are, if it is supported by the device, a higher resolution than the initial settings you provided so that there is minimal loss in resolution after the effect's cropping is applied.

Call [VideoDeviceController.SetMediaStreamPropertiesAsync](#) to set the new encoding properties. Before setting the new properties, use the member variable to store the initial encoding properties so that you can change the settings back when you disable the effect.

If the video stabilization effect must crop the output video, the recommended output encoding properties will be the size of the cropped video. This means that the output resolution will match the cropped video size. If you do not use the recommended output properties, the video will be scaled up to match the initial output size, which will result in a loss of visual fidelity.

Set the [Video](#) property of the **MediaEncodingProfile** object. Before setting the new properties, use the member variable to store the initial encoding properties so that you can change the settings back when you disable the effect.

C#

```
private async Task SetUpVideoStabilizationRecommendationAsync()
{
    // Get the recommendation from the effect based on our current input and
    // output configuration
    var recommendation =
        _videoStabilizationEffect.GetRecommendedStreamConfiguration(mediaCapture.VideoDeviceController, _encodingProfile.Video);

    // Handle the recommendation for the input into the effect, which can
    // contain a larger resolution than currently configured, so cropping is
```

```

minimized
    if (recommendation.InputProperties != null)
    {
        // Back up the current input properties from before VS was activated
        _inputPropertiesBackup =
mediaCapture.VideoDeviceController.GetMediaStreamProperties(MediaStreamType.
VideoRecord) as VideoEncodingProperties;

        // Set the recommendation from the effect (a resolution higher than
the current one to allow for cropping) on the input
        await
mediaCapture.VideoDeviceController.SetMediaStreamPropertiesAsync(MediaStream
Type.VideoRecord, recommendation.InputProperties);
        await
mediaCapture.VideoDeviceController.SetMediaStreamPropertiesAsync(MediaStream
Type.VideoPreview, recommendation.InputProperties);
    }

    // Handle the recommendations for the output from the effect
    if (recommendation.OutputProperties != null)
    {
        // Back up the current output properties from before VS was
activated
        _outputPropertiesBackup = _encodingProfile.Video;

        // Apply the recommended encoding profile for the output
        _encodingProfile.Video = recommendation.OutputProperties;
    }
}

```

Handle the video stabilization effect being disabled

The system may automatically disable the video stabilization effect if the pixel throughput is too high for the effect to handle or if it detects that the effect is running slowly. If this occurs, the `EnabledChanged` event is raised. The **VideoStabilizationEffect** instance in the *sender* parameter indicates the new state of the effect, enabled or disabled. The [VideoStabilizationEffectEnabledChangedEventArgs](#) has a [VideoStabilizationEffectEnabledChangedReason](#) value indicating why the effect was enabled or disabled. Note that this event is also raised if you programmatically enable or disable the effect, in which case the reason will be **Programmatic**.

Typically, you would use this event to adjust your app's UI to indicate the current status of video stabilization.

C#

```

private async void
VideoStabilizationEffect_EnabledChanged(VideoStabilizationEffect sender,
VideoStabilizationEffectEnabledChangedEventArgs args)

```



```

{
    await Dispatcher.RunAsync(CoreDispatcherPriority.Normal, () =>
    {
        // Update your UI to reflect the change in status
        ShowMessageToUser("video stabilization status: " + sender.Enabled +
            ". Reason: " + args.Reason);
    });
}

```

Clean up the video stabilization effect

To clean up the video stabilization effect, call [RemoveEffectAsync](#) to remove the effect from the video pipeline. If the member variables containing the initial encoding properties are not null, use them to restore the encoding properties. Finally, remove the **EnabledChanged** event handler and set the effect to null.

C#

```

// Clear all effects in the pipeline
await mediaCapture.RemoveEffectAsync(_videoStabilizationEffect);

// If backed up settings (stream properties and encoding profile) exist,
restore them and clear the backups
if (_inputPropertiesBackup != null)
{
    await
mediaCapture.VideoDeviceController.SetMediaStreamPropertiesAsync(MediaStream
Type.VideoRecord, _inputPropertiesBackup);
    _inputPropertiesBackup = null;
}

if (_outputPropertiesBackup != null)
{
    _encodingProfile.Video = _outputPropertiesBackup;
    _outputPropertiesBackup = null;
}

_videoStabilizationEffect.EnabledChanged -=
VideoStabilizationEffect_EnabledChanged;

_videoStabilizationEffect = null;

```

Related topics

- [Camera](#)
- [Basic photo, video, and audio capture with MediaCapture](#)

Effects for analyzing camera frames

Article • 10/20/2022

This article describes how to use the [SceneAnalysisEffect](#) and the [FaceDetectionEffect](#) to analyze the content of the media capture preview stream.

Scene analysis effect

The [SceneAnalysisEffect](#) analyzes the video frames in the media capture preview stream and recommends processing options to improve the capture result. Currently, the effect supports detecting whether the capture would be improved by using High Dynamic Range (HDR) processing.

If the effect recommends using HDR, you can do this in the following ways:

- Use the [AdvancedPhotoCapture](#) class to capture photos using the Windows built-in HDR processing algorithm. For more information, see [High Dynamic Range \(HDR\) photo capture](#).
- Use the [HdrVideoControl](#) to capture video using the Windows built-in HDR processing algorithm. For more information, see [Capture device controls for video capture](#).
- Use the [VariablePhotoSequenceControl](#) to capture a sequence of frames that you can then composite using a custom HDR implementation. For more information, see [Variable photo sequence](#).

Scene analysis namespaces

To use scene analysis, your app must include the following namespaces in addition to the namespaces required for basic media capture.

C#

```
using Windows.Media.Core;  
using Windows.Media.Devices;
```

Initialize the scene analysis effect and add it to the preview stream

Video effects are implemented using two APIs, an effect definition, which provides settings that the capture device needs to initialize the effect, and an effect instance, which can be used to control the effect. Since you may want to access the effect instance from multiple places within your code, you should typically declare a member variable to hold the object.

C#

```
private SceneAnalysisEffect _sceneAnalysisEffect;
```

In your app, after you have initialized the **MediaCapture** object, create a new instance of **SceneAnalysisEffectDefinition**.

Register the effect with the capture device by calling **AddVideoEffectAsync** on your **MediaCapture** object, providing the **SceneAnalysisEffectDefinition** and specifying **MediaStreamType.VideoPreview** to indicate that the effect should be applied to the video preview stream, as opposed to the capture stream. **AddVideoEffectAsync** returns an instance of the added effect. Because this method can be used with multiple effect types, you must cast the returned instance to a **SceneAnalysisEffect** object.

To receive the results of the scene analysis, you must register a handler for the **SceneAnalyzed** event.

Currently, the scene analysis effect only includes the high dynamic range analyzer. Enable HDR analysis by setting the effect's **HighDynamicRangeControl.Enabled** to true.

C#

```
// Create the definition
var definition = new SceneAnalysisEffectDefinition();

// Add the effect to the video record stream
_sceneAnalysisEffect = (SceneAnalysisEffect)await
_mediaCapture.AddVideoEffectAsync(definition, MediaStreamType.VideoPreview);

// Subscribe to notifications about scene information
_sceneAnalysisEffect.SceneAnalyzed += SceneAnalysisEffect_SceneAnalyzed;

// Enable HDR analysis
_sceneAnalysisEffect.HighDynamicRangeAnalyzer.Enabled = true;
```

Implement the SceneAnalyzed event handler

The results of the scene analysis are returned in the **SceneAnalyzed** event handler. The **SceneAnalyzedEventArgs** object passed into the handler has a

SceneAnalysisEffectFrame object which has a **HighDynamicRangeOutput** object. The **Certainty** property of the high dynamic range output provides a value between 0 and 1.0 where 0 indicates that HDR processing would not help improve the capture result and 1.0 indicates that HDR processing would help. You can decide the threshold point at which you want to use HDR or show the results to the user and let the user decide.

C#

```
private void SceneAnalysisEffect_SceneAnalyzed(SceneAnalysisEffect sender,
SceneAnalyzedEventArgs args)
{
    double hdrCertainty = args.ResultFrame.HighDynamicRange.Certainty;

    // Certainty value is between 0.0 and 1.0
    if(hdrCertainty > MyCertaintyCap)
    {
        ShowMessageToUser("Enabling HDR capture is recommended.");
    }
}
```

The **HighDynamicRangeOutput** object passed into the handler also has a **FrameControllers** property which contains suggested frame controllers for capturing a variable photo sequence for HDR processing. For more information, see [Variable photo sequence](#).

Clean up the scene analysis effect

When your app is done capturing, before disposing of the **MediaCapture** object, you should disable the scene analysis effect by setting the effect's **HighDynamicRangeAnalyzer.Enabled** property to false and unregister your **SceneAnalyzed** event handler. Call **MediaCapture.ClearEffectsAsync**, specifying the video preview stream since that was the stream to which the effect was added. Finally, set your member variable to null.

C#

```
// Disable detection
_sceneAnalysisEffect.HighDynamicRangeAnalyzer.Enabled = false;

_sceneAnalysisEffect.SceneAnalyzed -= SceneAnalysisEffect_SceneAnalyzed;

// Remove the effect from the preview stream
await _mediaCapture.ClearEffectsAsync(MediaStreamType.VideoPreview);

// Clear the member variable that held the effect instance
_sceneAnalysisEffect = null;
```

Face detection effect

The [FaceDetectionEffect](#) identifies the location of faces within the media capture preview stream. The effect allows you to receive a notification whenever a face is detected in the preview stream and provides the bounding box for each detected face within the preview frame. On supported devices, the face detection effect also provides enhanced exposure and focus on the most important face in the scene.

Face detection namespaces

To use face detection, your app must include the following namespaces in addition to the namespaces required for basic media capture.

C#

```
using Windows.Media.Core;
```

Initialize the face detection effect and add it to the preview stream

Video effects are implemented using two APIs, an effect definition, which provides settings that the capture device needs to initialize the effect, and an effect instance, which can be used to control the effect. Since you may want to access the effect instance from multiple places within your code, you should typically declare a member variable to hold the object.

C#

```
FaceDetectionEffect _faceDetectionEffect;
```

In your app, after you have initialized the **MediaCapture** object, create a new instance of [FaceDetectionEffectDefinition](#). Set the [DetectionMode](#) property to prioritize faster face detection or more accurate face detection. Set [SynchronousDetectionEnabled](#) to specify that incoming frames are not delayed waiting for face detection to complete as this can result in a choppy preview experience.

Register the effect with the capture device by calling [AddVideoEffectAsync](#) on your **MediaCapture** object, providing the **FaceDetectionEffectDefinition** and specifying [MediaStreamType.VideoPreview](#) to indicate that the effect should be applied to the video preview stream, as opposed to the capture stream. **AddVideoEffectAsync** returns

an instance of the added effect. Because this method can be used with multiple effect types, you must cast the returned instance to a [FaceDetectionEffect](#) object.

Enable or disable the effect by setting the [FaceDetectionEffect.Enabled](#) property. Adjust how often the effect analyzes frames by setting the [FaceDetectionEffect.DesiredDetectionInterval](#) property. Both of these properties can be adjusted while media capture is ongoing.

C#

```
// Create the definition, which will contain some initialization settings
var definition = new FaceDetectionEffectDefinition();

// To ensure preview smoothness, do not delay incoming samples
definition.SynchronousDetectionEnabled = false;

// In this scenario, choose detection speed over accuracy
definition.DetectionMode = FaceDetectionMode.HighPerformance;

// Add the effect to the preview stream
_faceDetectionEffect = (FaceDetectionEffect)await
_mediaCapture.AddVideoEffectAsync(definition, MediaStreamType.VideoPreview);

// Choose the shortest interval between detection events
_faceDetectionEffect.DesiredDetectionInterval =
    TimeSpan.FromMilliseconds(33);

// Start detecting faces
_faceDetectionEffect.Enabled = true;
```

Receive notifications when faces are detected

If you want to perform some action when faces are detected, such as drawing a box around detected faces in the video preview, you can register for the [FaceDetected](#) event.

C#

```
// Register for face detection events
_faceDetectionEffect.FaceDetected += FaceDetectionEffect_FaceDetected;
```

In the handler for the event, you can get a list of all faces detected in a frame by accessing the [FaceDetectionEffectFrame.DetectedFaces](#) property of the [FaceDetectedEventArgs](#). The [FaceBox](#) property is a [BitmapBounds](#) structure that describes the rectangle containing the detected face in units relative to the preview

stream dimensions. To view sample code that transforms the preview stream coordinates into screen coordinates, see the [face detection UWP sample](#).

C#

```
private void FaceDetectionEffect_FaceDetected(FaceDetectionEffect sender,
FaceDetectedEventArgs args)
{
    foreach (Windows.Media.FaceAnalysis.DetectedFace face in
args.ResultFrame.DetectedFaces)
    {
        BitmapBounds faceRect = face.FaceBox;

        // Draw a rectangle on the preview stream for each face
    }
}
```

Clean up the face detection effect

When your app is done capturing, before disposing of the **MediaCapture** object, you should disable the face detection effect with **FaceDetectionEffect.Enabled** and unregister your **FaceDetected** event handler if you previously registered one. Call **MediaCapture.ClearEffectsAsync**, specifying the video preview stream since that was the stream to which the effect was added. Finally, set your member variable to null.

C#

```
// Disable detection
_faceDetectionEffect.Enabled = false;

// Unregister the event handler
_faceDetectionEffect.FaceDetected -= FaceDetectionEffect_FaceDetected;

// Remove the effect from the preview stream
await _mediaCapture.ClearEffectsAsync(MediaStreamType.VideoPreview);

// Clear the member variable that held the effect instance
_faceDetectionEffect = null;
```

Check for focus and exposure support for detected faces

Not all devices have a capture device that can adjust its focus and exposure based on detected faces. Because face detection consumes device resources, you may only want to enable face detection on devices that can use the feature to enhance capture. To see if face-based capture optimization is available, get the **VideoDeviceController** for your initialized **MediaCapture** and then get the video device controller's

[RegionsOfInterestControl](#). Check to see if the [MaxRegions](#) supports at least one region. Then check to see if either [AutoExposureSupported](#) or [AutoFocusSupported](#) are true. If these conditions are met, then the device can take advantage of face detection to enhance capture.

C#

```
var regionsControl =  
_mediaCapture.VideoDeviceController.RegionsOfInterestControl;  
bool faceDetectionFocusAndExposureSupported =  
    regionsControl.MaxRegions > 0 &&  
    (regionsControl.AutoExposureSupported ||  
regionsControl.AutoFocusSupported);
```

Related topics

- [Camera](#)
- [Basic photo, video, and audio capture with MediaCapture](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Variable photo sequence

Article • 10/20/2022

This article shows you how to capture a variable photo sequence, which allows you to capture multiple frames of images in rapid succession and configure each frame to use different focus, flash, ISO, exposure, and exposure compensation settings. This feature enables scenarios like creating High Dynamic Range (HDR) images.

If you want to capture HDR images but don't want to implement your own processing algorithm, you can use the [AdvancedPhotoCapture](#) API to use the HDR capabilities built-in to Windows. For more information, see [High Dynamic Range \(HDR\) photo capture](#).

ⓘ Note

This article builds on concepts and code discussed in [Basic photo, video, and audio capture with MediaCapture](#), which describes the steps for implementing basic photo and video capture. It is recommended that you familiarize yourself with the basic media capture pattern in that article before moving on to more advanced capture scenarios. The code in this article assumes that your app already has an instance of MediaCapture that has been properly initialized.

Set up your app to use variable photo sequence capture

In addition to the namespaces required for basic media capture, implementing a variable photo sequence capture requires the following namespaces.

C#

```
using Windows.Media.Capture.Core;  
using Windows.Media.Devices.Core;
```

Declare a member variable to store the [VariablePhotoSequenceCapture](#) object, which is used to initiate the photo sequence capture. Declare an array of [SoftwareBitmap](#) objects to store each captured image in the sequence. Also, declare an array to store the [CapturedFrameControlValues](#) object for each frame. This can be used by your image processing algorithm to determine what settings were used to capture each frame.

Finally, declare an index that will be used to track which image in the sequence is currently being captured.

C#

```
VariablePhotoSequenceCapture _photoSequenceCapture;  
SoftwareBitmap[] _images;  
CapturedFrameControlValues[] _frameControlValues;  
int _photoIndex;
```

Prepare the variable photo sequence capture

After you have initialized your [MediaCapture](#), make sure that variable photo sequences are supported on the current device by getting an instance of the [VariablePhotoSequenceController](#) from the media capture's [VideoDeviceController](#) and checking the [Supported](#) property.

C#

```
var varPhotoSeqController =  
_mediaCapture.VideoDeviceController.VariablePhotoSequenceController;  
  
if (!varPhotoSeqController.Supported)  
{  
    ShowMessageToUser("Variable Photo Sequence is not supported");  
    return;  
}
```

Get a [FrameControlCapabilities](#) object from the variable photo sequence controller. This object has a property for every setting that can be configured per frame of a photo sequence. These include:

- [Exposure](#)
- [ExposureCompensation](#)
- [Flash](#)
- [Focus](#)
- [IsoSpeed](#)
- [PhotoConfirmation](#)

This example will set a different exposure compensation value for each frame. To verify that exposure compensation is supported for photo sequences on the current device, check the [Supported](#) property of the [FrameExposureCompensationCapabilities](#) object accessed through the [ExposureCompensation](#) property.

C#

```
var frameCapabilities = varPhotoSeqController.FrameCapabilities;

if (!frameCapabilities.ExposureCompensation.Supported)
{
    ShowMessageToUser("EVCompensation is not supported in FrameController");
    return;
}
```

Create a new [FrameController](#) object for each frame you want to capture. This example captures three frames. Set the values for the controls you want to vary for each frame. Then, clear the [DesiredFrameControllers](#) collection of the [VariablePhotoSequenceController](#) and add each frame controller to the collection.

C#

```
var frame0 = new FrameController();
var frame1 = new FrameController();
var frame2 = new FrameController();

frame0.ExposureCompensationControl.Value = -1.0f;
frame1.ExposureCompensationControl.Value = 0.0f;
frame2.ExposureCompensationControl.Value = 1.0f;

varPhotoSeqController.DesiredFrameControllers.Clear();
varPhotoSeqController.DesiredFrameControllers.Add(frame0);
varPhotoSeqController.DesiredFrameControllers.Add(frame1);
varPhotoSeqController.DesiredFrameControllers.Add(frame2);
```

Create an [ImageEncodingProperties](#) object to set the encoding you want to use for the captured images. Call the static method [MediaCapture.PrepareVariablePhotoSequenceCaptureAsync](#), passing in the encoding properties. This method returns a [VariablePhotoSequenceCapture](#) object. Finally, register event handlers for the [PhotoCaptured](#) and [Stopped](#) events.

C#

```
try
{
    var imageEncodingProperties = ImageEncodingProperties.CreateJpeg();

    _photoSequenceCapture = await
        _mediaCapture.PrepareVariablePhotoSequenceCaptureAsync(imageEncodingProperties);

    _photoSequenceCapture.PhotoCaptured += OnPhotoCaptured;
    _photoSequenceCapture.Stopped += OnStopped;
}
```

```
catch (Exception ex)
{
    ShowMessageToUser("Exception in PrepareVariablePhotoSequence: " +
ex.Message);
}
```

Start the variable photo sequence capture

To start the capture of the variable photo sequence, call [VariablePhotoSequenceCapture.StartAsync](#). Be sure to initialize the arrays for storing the captured images and frame control values and set the current index to 0. Set your app's recording state variable and update your UI to disable starting another capture while this capture is in progress.

```
C#

private async void StartPhotoCapture()
{
    _images = new SoftwareBitmap[3];
    _frameControlValues = new CapturedFrameControlValues[3];
    _photoIndex = 0;
    _isRecording = true;

    await _photoSequenceCapture.StartAsync();
}
```

Receive the captured frames

The [PhotoCaptured](#) event is raised for each captured frame. Save the frame control values and captured image for the frame and then increment the current frame index. This example shows how to get a [SoftwareBitmap](#) representation of each frame. For more information on using [SoftwareBitmap](#), see [Imaging](#).

```
C#

void OnPhotoCaptured(VariablePhotoSequenceCapture s,
VariablePhotoCapturedEventArgs args)
{
    _images[_photoIndex] = args.Frame.SoftwareBitmap;
    _frameControlValues[_photoIndex] = args.CapturedFrameControlValues;
    _photoIndex++;
}
```